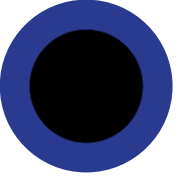




TECH2

Week 1 Workshop

Finding Your Way Around VS Code



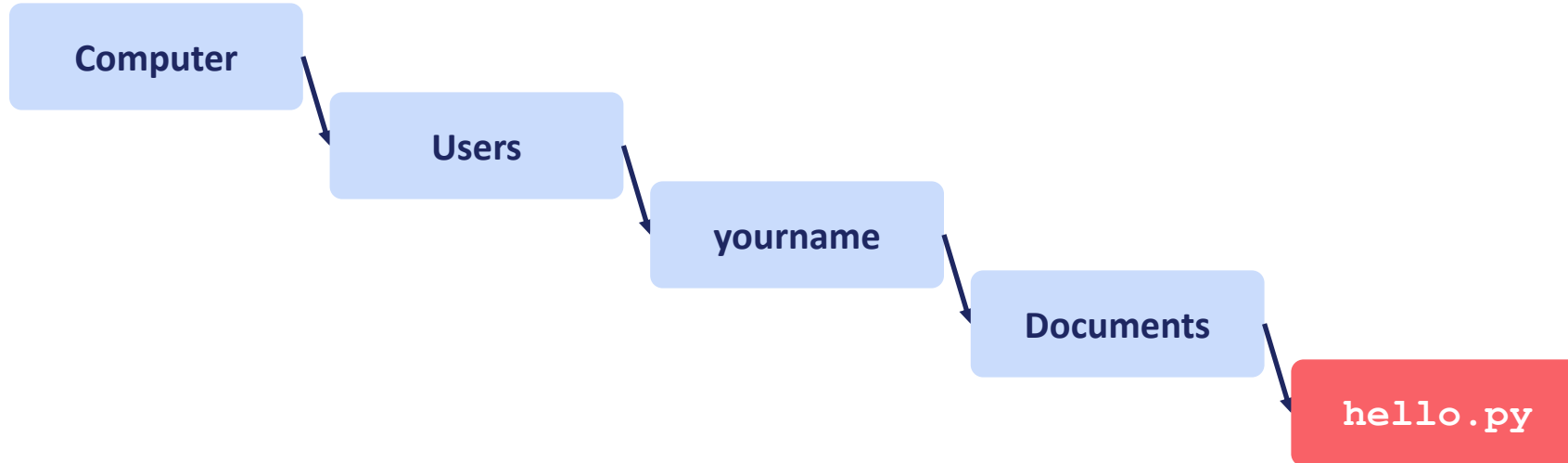
Today's Plan

- Understand files, folders, and paths on your computer
- Get comfortable typing simple commands in a terminal
- Build your course folder for the semester
- Write and run your first Python script
- Create your first Jupyter notebook

Goal for today: by next week, you can download material and get straight to work — no time lost looking for files.



Files, Folders, and Paths

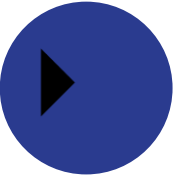


A path is just directions through the tree — one folder inside another.

Windows: `C:\Users\yourname\Documents\hello.py`

Mac: `/Users/yourname/Documents/hello.py`

Windows uses backslashes (\) — Mac (and Linux) use forward slashes (/).



What Is the Terminal?

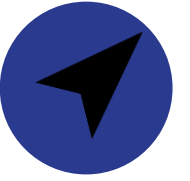
- A text window where you type instructions instead of clicking icons (command-line interface to your computer)
- Why we use it: precise, fast once learned, and how real coding and data work actually happens
- Every programmer and data professional uses one — a core skill, not a workaround
- Mac's Terminal is Unix-based but Windows' shell speaks different command than Unix → Use Git Bash on Windows

Windows

Start Menu → search “Git Bash” → open

Mac

Cmd+Space → type “Terminal” → open



Your Working Directory

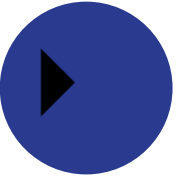
- “Where you are currently standing”
- **Technical:** The active folder on your computer where a program, command-line shell, or script runs and looks for files by default
- **Practical:** Simply the folder that you’re currently “inside” while working
- You can move around — but you're always somewhere
- Rule #1: always know where you are before you act
- When you run Python, it will look for files in the working directory — if the file is not there, you must tell Python the path to where the file lives

Windows

Right-click on file → Copy as path

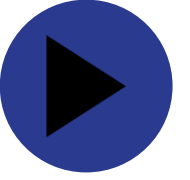
Mac

Hold Option → Right-click on file → Copy ... as Pathname



Four Commands to Know

Command	What it does
<code>pwd</code>	Show current location
<code>ls</code>	List what's in the current folder
<code>mkdir name</code>	Create a new folder
<code>cd name</code> <code>cd ..</code> <code>cd</code>	Move into a folder Move up one level Move back to default root directory



Exercise 1 — Follow Along

- 1 Open Git Bash / Terminal
- 2 Run `pwd` — where are you?
- 3 Run `ls` — what's here?
- 4 Create a folder: `mkdir practice`
- 5 Move into it: `cd practice`
- 6 Run `pwd` again — did it change?

Tip: open File Explorer/Finder alongside it — does what you see there match what the terminal shows?



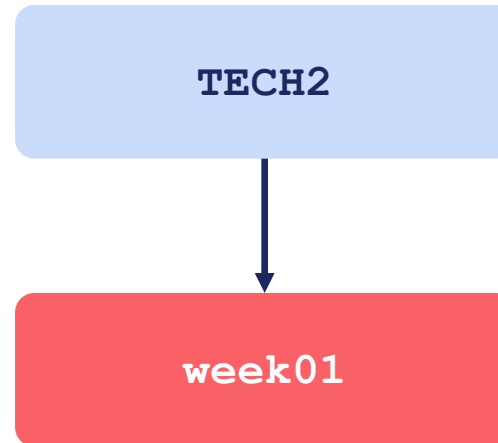
Exercise 2 — Find Your Own Path

- 1 In File Explorer (Win) or Finder (Mac), find a folder you use often
- 2 Windows: right-click → Copy as path. Mac: hold Option, right-click → Copy ... as Pathname
- 3 In Git Bash, type `cd` (with a space), then paste the path (Windows: Shift+Insert. Mac: Cmd+V)
- 4 Press Enter, then confirm with `pwd`
- 5 Run `ls` to see what's inside

Tip: On Windows, change backslashes to forward slashes, or simply enclose file paths in quotation marks.

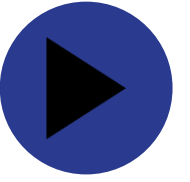


Build Your Course Folder



This is a folder inside a folder — TECH2 is your course, week01 is this week's workspace.

Note: You'll create a new week folder like this, inside TECH2, every week of the course.

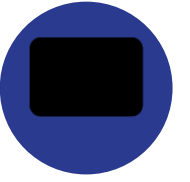


Exercise 3 — Create TECH2/week01

Create TECH2 folder in your Documents folder (or another sensible location on your computer):

```
cd Documents
mkdir TECH2
cd TECH2
mkdir week01
cd week01
pwd
```

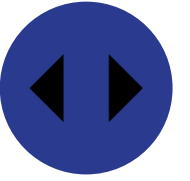
Tip: Instead of the terminal, you can use File explorer / Finder to create the folder. The important thing is that you know how to navigate to this folder!



Opening VS Code: Projects & Working Directory

- 1 Open VS Code
 - 2 File → Open Folder → select the TECH2 folder you just built
 - 3 Look at the Explorer sidebar — it now mirrors exactly what `pwd`/`ls` showed you
- Opening a folder like this is called opening a “project”
 - Whichever folder you open becomes VS Code's working directory
 - Same idea as `pwd` — just set visually instead of typed

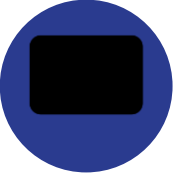
Tip: You can also use VS Code to create new folders in your project.



Your First Script

- 1 In the Explorer, right-click week01 → New File
- 2 Name it hello.py
- 3 VS Code will ask to install the Python extension

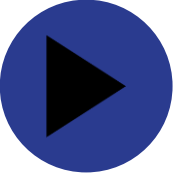
Note: Say yes to the extension prompt when it appears — that's expected, and exactly when we need it.



Check Your Interpreter

- Bottom-right corner of VS Code shows the active interpreter
- This is the Python environment your code will run in
- Click it to change — today, use the environment (miniforge) you set up earlier

***Tip:** Always check that you have selected the correct interpreter before executing your code.*



Exercise 4 — Write & Run Your Script

- 1 In hello.py, type the following code:

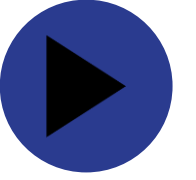
```
# my first python script  
price = 19.99  
quantity = 3  
print("Total:", price*quantity)
```

- 2 Click the ► Play button to run hello.py
- 3 Watch the output appear in the terminal in VS Code
- 4 Run the script again. Did anything carry over from the last run?



Your First Notebook

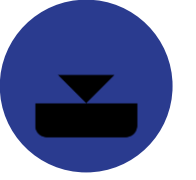
- 1 New File → hello.ipynb
- 2 VS Code will ask to install the Jupyter extension (say yes to all extension prompts)
- 3 Select the same environment as your notebook's kernel



Exercise 5 — Build Your Notebook

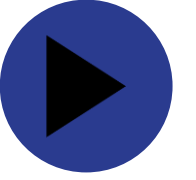
- 1 Add a markdown cell (e.g., a title, a short bullet list)
- 2 Add a code cell reusing lines from your script
- 3 Run both cells
- 4 Add a third line and run again — did the old code disappear?

Note: *variables stick around between cells — unlike a script's fresh start every run.*



Download This Week's Files

- Course material lives on GitHub — that is always the current version
- Each week, a link to that week's folder is posted on Canvas
- GitHub has no “download folder” button — you download the files one at a time
- You'll repeat this step every week of the course



Exercise 6 — Download & Place Your Files

- 1 Open Canvas and click this week's link — it opens the week01 folder on GitHub
- 2 Click `example_script.py`, then the download button. Do the same for `example_notebook.ipynb`
- 3 Move both into your week01 folder, next to the `hello.py` you wrote
- 4 Confirm in Explorer — are they really there?



What You've Built Today

- TECH2/week01 — your course folder, created by hand
- This week's reference files — downloaded and placed correctly
- hello.py — your first script
- hello.ipynb — your first notebook

Next week: real Python coding!
Come with everything installed and ready to go